

Introduction à Docker Compose : Simplifier l'Orchestration Multi-Conteneurs

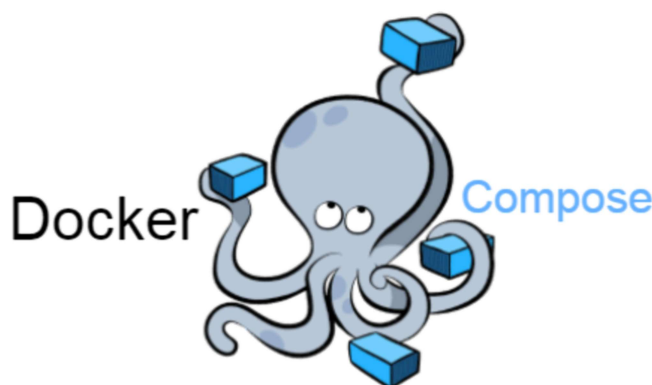
Pourquoi Docker Compose ?

Docker Compose est un outil essentiel pour la gestion d'applications composées de plusieurs conteneurs. Contrairement à l'approche consistant à regrouper toutes les fonctionnalités dans un seul conteneur monolithique, il est préférable de manipuler plusieurs conteneurs de tailles réduites, chacun étant responsable d'une fonction spécifique (par exemple, serveur web, base de données, etc.).

Cependant, gérer manuellement ces conteneurs peut vite devenir complexe :

- Démarrer et arrêter les conteneurs un par un est fastidieux.
- Les relations entre conteneurs (réseaux, volumes, etc.) sont difficiles à maintenir.

C'est ici que Docker Compose intervient



Qu'est-ce que Docker Compose ?

Docker Compose est un outil développé par Docker qui permet de définir et d'exécuter des applications multi-conteneurs de manière simple et cohérente. Il offre :

- **Une syntaxe évoluée** pour orchestrer plusieurs conteneurs.
- **Une modularité accrue**, permettant d'organiser chaque brique de l'application (code, base de données, serveur web, etc.) dans des conteneurs distincts.
- **Un confort d'utilisation**, en simplifiant les déploiements complexes.
- **L'utilisation des fichiers de configuration écrits en YAML (*Yet Another Markup Language*)** pour décrire l'architecture des services et leurs dépendances.

Les trois étapes principales de Docker Compose

L'utilisation de **Compose** se résume à un processus en trois étapes :

1. Définir l'environnement de l'application

Utiliser un fichier **Dockerfile** pour définir l'image Docker de chaque service. Cela garantit que l'environnement est reproductible sur n'importe quelle machine.

2.Décrire les services

Créer un fichier **docker-compose.yml** pour définir les différents services qui composent l'application, leurs configurations (volumes, réseaux, ports, etc.) et leurs relations.

3.Exécuter l'application

Lancer la commande **docker-compose up** pour déployer l'ensemble de l'application.

Fonctionnalités de Docker Compose

Construction et démarrage automatique des services : Compose extrait automatiquement les images, construit des conteneurs et les démarre.

Configuration centralisée : Les liens entre conteneurs, les volumes partagés, et d'autres options Docker sont configurés dans un seul fichier.

Exécution en avant-plan ou arrière-plan : Compose peut exécuter les conteneurs en mode interactif (avant-plan) ou en mode détaché (arrière-plan).

Affichage des logs agrégés : En mode avant-plan, les logs de tous les conteneurs sont affichés de manière centralisée, facilitant le débogage.

Docker compose: Installation

Avant de procéder à l'installation, vérifiez si Docker Compose est déjà installé sur votre machine en exécutant la commande suivante :

```
docker compose version
```

Si la commande retourne la version de Docker Compose, cela signifie qu'il est déjà installé. Sinon, suivez les étapes ci-dessous pour l'installer.

Prérequis :

Avant de commencer, assurez-vous de disposer des éléments suivants :

- Une machine Ubuntu 22.04.
- Docker installé sur votre machine.

Étapes d'installation de Docker Compose :

1. Utilisez les commandes suivantes pour télécharger Docker Compose:

```
mkdir -p ~/.docker/cli-plugins/  
  
curl -SL https://github.com/docker/compose/releases/download/v2.3.3/docker-compose-  
linux-x86_64 -o ~/.docker/cli-plugins/docker-compose
```

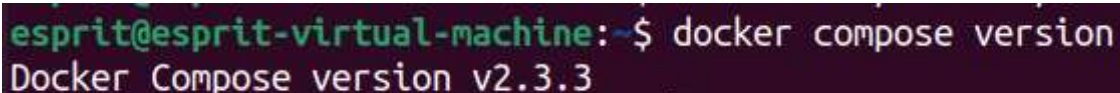
2. Après le téléchargement, vous devez définir les bonnes autorisations pour rendre la commande `docker compose` exécutable :

```
chmod +x ~/.docker/cli-plugins/docker-compose
```

3. Une fois l'installation terminée, vérifiez que Docker Compose a été correctement installé en exécutant la commande suivante :

```
docker compose version
```

Vous verrez un résultat similaire à celui-ci :



```
esprit@esprit-virtual-machine:~$ docker compose version  
Docker Compose version v2.3.3
```

Si cette sortie apparaît, cela signifie que Docker Compose a été installé avec succès et que vous pouvez maintenant utiliser la commande `docker compose` pour gérer vos conteneurs Docker via des fichiers `docker-compose.yml`.

Remarque :

À partir de Docker Compose v2, Docker a migré vers une commande plugin CLI docker compose au lieu de l'ancienne commande docker-compose. La principale différence réside dans le retrait du tiret dans la commande.

Docker Compose : Commandes de base

Voici les principales commandes de Docker Compose à connaître pour travailler efficacement avec des stacks de conteneurs:

- **Démarrer et exécuter l'application complète :**

Cette commande démarre les conteneurs définis dans le fichier docker-compose.yml et exécute l'application entière.

```
docker compose up
```

- **Démarrer et exécuter l'application en arrière-plan :**

Utilisée pour démarrer l'application en mode détaché (en arrière-plan), ce qui permet de continuer à utiliser le terminal pour d'autres commandes.

```
docker compose up -d
```

- **Afficher le groupe de conteneurs (en cours d'exécution ou arrêtés) :**

Cette commande affiche la liste des conteneurs associés à l'application, qu'ils soient en cours d'exécution ou arrêtés.

```
docker compose ps
```

- **Afficher la configuration de l'application (tous les paramètres et configurations) :**

Permet de visualiser la configuration complète de l'application Docker Compose, y compris les services, volumes et réseaux définis dans le fichier docker-compose.yml.

```
docker compose config
```

- **Afficher les logs des conteneurs :**

Cette commande permet de consulter les journaux des conteneurs, ce qui est utile pour déboguer ou surveiller l'activité de l'application.

```
docker compose logs
```

- **Arrêter une stack Docker Compose(l'application) sans supprimer les ressources créées :**

Permet d'arrêter les conteneurs en cours d'exécution sans supprimer les ressources créées (comme les volumes ou les réseaux).

```
docker compose stop
```

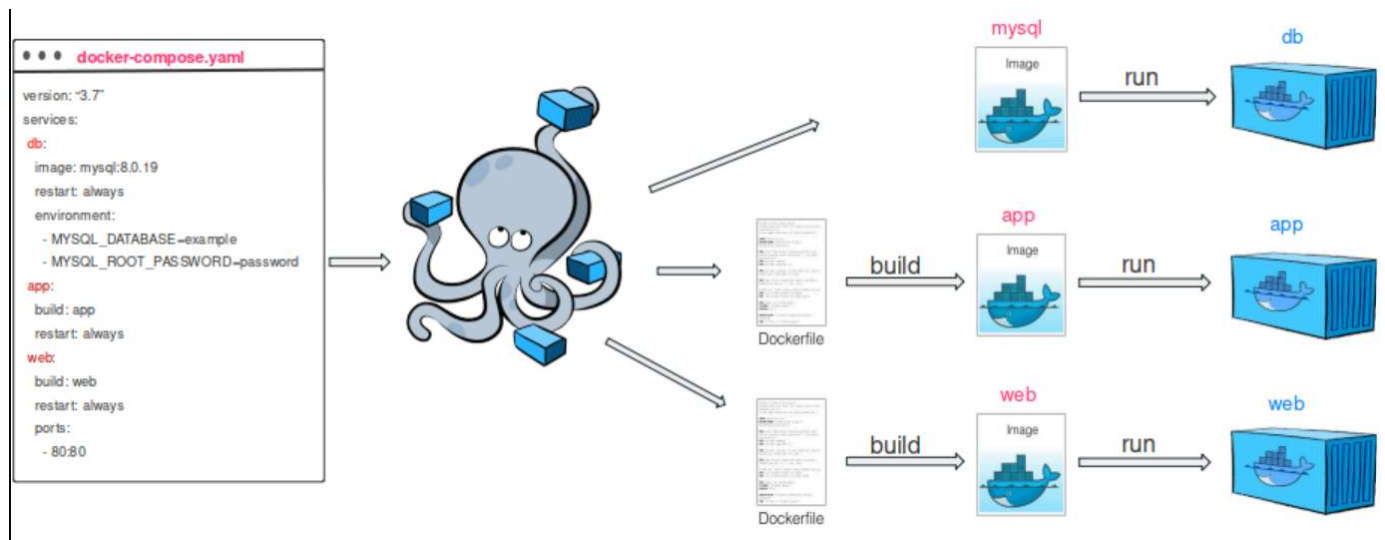
- **Supprimer une stack Docker Compose (l'application) ainsi que toutes les ressources associées :**

Cette commande arrête les conteneurs et supprime les ressources créées par Docker Compose, y compris les réseaux, volumes et conteneurs.

```
docker compose down
```

Maîtriser la structure du fichier docker-compose.yml

Le fichier docker-compose.yml est un outil essentiel pour orchestrer des applications multi-conteneurs avec Docker. Il permet de définir, configurer et gérer plusieurs services dans un environnement Docker de manière fluide et efficace. En utilisant ce fichier, vous pouvez spécifier des paramètres cruciaux pour chaque service, tels que l'image Docker à utiliser, les variables d'environnement, les volumes partagés, les ports exposés, et même la politique de redémarrage des conteneurs.



Voici les principales directives utilisées dans un fichier docker-compose.yml :

version : La première directive du fichier est `version`, qui permet de spécifier à Docker Compose quelle version yml à utiliser

Exemple :

```
version: '3.8'
```

services : Sous la directive `services`, on définit l'ensemble des conteneurs qui doivent être créés. Chaque service représente un conteneur Docker à lancer. Ces services peuvent être liés entre eux, partagent souvent des réseaux ou des volumes et sont configurés de manière spécifique.

Exemple :

```
services:
  web:
    image: nginx
  db:
    image: postgres
```

image : Permet de spécifier quelle image Docker doit être utilisée pour créer un conteneur.

Exemple :

```
services:
  web:
    image: nginx:latest
```

container_name : Permet de définir le nom du conteneur. Par défaut, Docker Compose génère un nom aléatoire, mais cette option permet de spécifier un nom fixe pour plus de clarté.

Exemple :

```
services:
  web:
    container_name: mon_conteneur_nginx
```

environment : Les variables d'environnement sont définies sous la clé environment. Cela permet de passer des configurations spécifiques au conteneur au moment de son lancement.

Exemple :

```
services:
  web:
    image: nginx
    environment:
      - NGINX_HOST=monsite.com
      - NGINX_PORT=80
```

restart : Permet de définir la politique de redémarrage d'un conteneur en cas d'arrêt. Par exemple, on peut définir que le conteneur redémarre automatiquement si le processus meurt .

```
services:
  web:
    image: nginx
    restart: always
```

Les options possibles pour restart sont :

- no : ne redémarre pas (par défaut).
- always : redémarre toujours.

- **on-failure** : redémarre uniquement en cas d'échec (vous pouvez aussi spécifier un nombre de tentatives).
- **unless-stopped** : redémarre sauf si le conteneur a été explicitement arrêté.

ports : Sous ports, on définit les correspondances des ports entre l'hôte et le conteneur. Cette option est importante pour rendre les services accessibles à l'extérieur du conteneur. Il est recommandé d'entourer les numéros de ports de guillemets pour éviter toute mauvaise interprétation.

Exemple :

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
```

Explication du format "**8080:80**" :

- **8080** : Le premier numéro de port (avant les deux-points) est le port de l'hôte, c'est-à-dire le port sur lequel l'application sera accessible à partir de l'extérieur de Docker (par exemple, dans un navigateur ou une autre machine).
- **80** : Le second numéro (après les deux-points) est le port à l'intérieur du conteneur. C'est le port sur lequel le service à l'intérieur du conteneur écoute (dans cet exemple, un serveur web Nginx qui écoute sur le port 80).

volumes : Les volumes permettent de créer une correspondance entre un répertoire de la machine hôte et un répertoire dans le conteneur. Cela est utile pour la persistance des données ou pour partager des fichiers entre le conteneur et l'hôte.

Exemple :

```
services:
  web:
    image: nginx
    volumes:
      - ./data:/usr/share/nginx/html
```

build : Si l'image du conteneur doit être construite à partir d'un fichier Dockerfile (plutôt que d'utiliser une image déjà existante), vous pouvez utiliser build.

Exemple :

```
services:
  web:
    build: .
```

Cela indique que Docker Compose doit construire l'image à partir du Dockerfile situé dans le répertoire courant (.).

depends_on : Permet de spécifier les services dont dépend un autre service pour son exécution. Par exemple, une application web pourrait dépendre d'une base de données.

Exemple :

```
services:
  web:
    image: webapp
    depends_on:
      - db
```

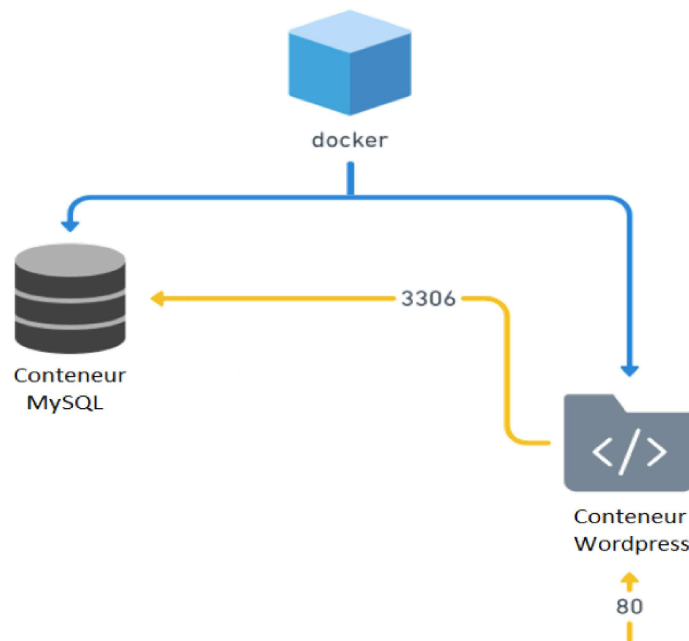
Exemple 1: Déploiement d'une application WordPress avec Docker Compose

Déploiement d'une application WordPress avec Docker Compose

Objectif : Créer et déployer un site web WordPress en utilisant Docker Compose, incluant :

- Une base de données MySQL.
- Le système de gestion de contenu WordPress connecté à cette base.

Architecture du projet:



1. Créer un répertoire nommé **wordapp** :

```
esprit@esprit-virtual-machine:~$ mkdir wordapp
esprit@esprit-virtual-machine:~$ cd wordapp/
```

2. Dans le répertoire **wordapp**, créer un fichier nommé **docker-compose.yml** avec le contenu suivant :

```

version: '3'
services:
  db2:
    image: mysql:5.7
    volumes:
      #volume:point de montage(in container)
      - db_data:/var/lib/mysql # Stocke les données MySQL dans un volume persistant
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: somewordpress # Mot de passe root
      MYSQL_DATABASE: wordpress # Nom de la base de données
      MYSQL_USER: wordpress # Nom de l'utilisateur
      MYSQL_PASSWORD: wordpress # Mot de passe de l'utilisateur
    networks:
      - my_network # Attache le service à un réseau

  wordpress2:
    depends_on:
      - db2 # S'assure que la base de données démarre avant WordPress
    image: wordpress:latest
    ports:
      - "8082:80" # Expose le port 80 du conteneur sur le port 8082 de l'hôte
    restart: always
    environment:
      WORDPRESS_DB_HOST: db2:3306 # Nom de l'hôte du serveur de la base de donnée hostname:port
      WORDPRESS_DB_USER: wordpress # Nom de l'utilisateur
      WORDPRESS_DB_PASSWORD: wordpress # Mot de passe de l'utilisateur
      WORDPRESS_DB_NAME: wordpress # Nom de la base de données
    networks:
      - my_network # Attache le service à un réseau

# Définition des volumes pour la persistance des données
volumes:
  db_data: {} # Stocke les données MySQL pour éviter leur perte

# Définition du réseau personnalisé
networks:
  my_network: {} # Réseau pour connecter MySQL et WordPress

```

Vous trouverez le contenu complet du fichier docker-compose.yml dans ce document.



docker-compose.yml.txt

✦ Explications :

- **Services** : Deux services sont définis, db (MySQL) et wordpress.
- **Volumes** : Les données MySQL sont stockées dans un volume nommé db_data pour éviter leur perte.
- **Ports** : Le port 80 du conteneur WordPress est exposé sur le port 8082 de ta machine. Cela signifie que tu accèderas à WordPress via <http://127.0.0.1:8082> (<http://127.0.0.1:8082>).
- **Environment** : Les variables définissent la connexion entre WordPress et la base de données.
- **Réseaux (networks)** : Un réseau personnalisé my_network est défini pour connecter les conteneurs.

3. Exécuter la commande **docker compose up -d** suivante pour démarrer les services en arrière-plan et lancer l'application WordPress :

```
esprit@esprit-virtual-machine:~/wordapp$ docker compose up -d
[+] Running 3/3
:: Network wordapp_my_network      Created
:: Container wordapp-wordpress2-1  Sta...
:: Container wordapp-db2-1         Started
```

4. Lister les conteneurs actifs avec la commande **docker ps** pour t'assurer que WordPress et MySQL sont bien déployés :

```
esprit@esprit-virtual-machine:~/wordapp$ docker compose ps
NAME                COMMAND                                SERVICE    STATUS    PORTS
wordapp-db2-1       "docker-entrypoint.s..."            db2        running   33060/tcp
wordapp-wordpress2-1 "docker-entrypoint.s..."            wordpress2 running   0.0.0.0:8082->80/tcp, :::8082->80/tcp
```

À ce stade :

- Un conteneur MySQL est créé pour la base de données.
- Un conteneur WordPress est créé et configuré pour se connecter à MySQL.

Les noms des conteneurs générés par Docker Compose se composent de plusieurs parties :

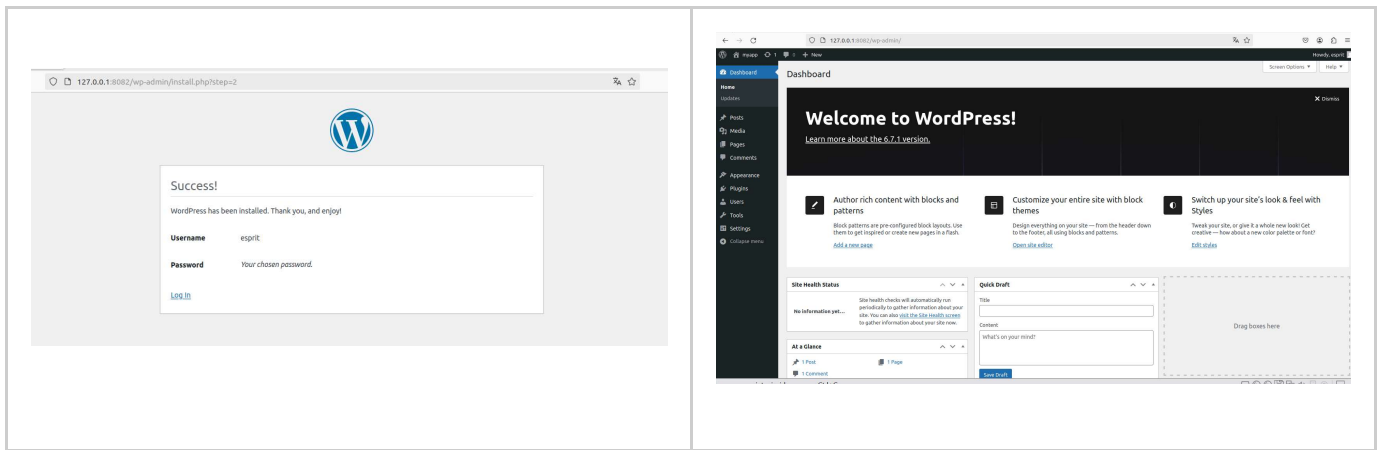
<nom_du_dossier>-<nom_du_service>-<numéro>

5. Ouvre ton navigateur et visite l'adresse suivante pour configurer ton site WordPress :

<http://127.0.0.1:8082> (<http://127.0.0.1:8082>).

The left screenshot shows the WordPress installation language selection screen. The language 'English (United States)' is selected. Below it, a list of other languages is visible, including Afrikaans, Azərbaycanca, বাংলা, العربية المغربية, বাংলা, Azərbaycanca dili, Беларуская мова, Български, বাংলা, Босански, Català, Cebuano, Čeština, Cymraeg, Dansk, Deutsch (Schweiz), Deutsch (Österreich), Deutsch (Schweiz, Du), Deutsch, and Deutsch (Sie). A 'Continue' button is at the bottom right.

The right screenshot shows the 'Welcome' screen of the WordPress installation process. It includes a 'Welcome' message and a section titled 'Information needed'. The form fields are filled with: Site Title: 'myapp', Username: 'esprit', Password: 'Strong', and Your Email: 'eya.triff@esprit.in'. There is a 'Show' button next to the password field. At the bottom, there is a checkbox for 'Search engine visibility' and an 'Install WordPress' button.



📌 Explication :

- 127.0.0.1 représente l'adresse locale de ta machine.
- 8082 est le port mappé au conteneur WordPress.

Résultat attendu : L'interface de configuration initiale de WordPress s'affiche. Suis les étapes pour créer ton site.

6.Vérifier la syntaxe et la validité de ton fichier Compose avec la commande **docker compose config** :

```
esprit@esprit-virtual-machine:~/wordapp$ docker compose config
name: wordapp
services:
  db2:
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_PASSWORD: wordpress
      MYSQL_ROOT_PASSWORD: somewordpress
      MYSQL_USER: wordpress
    image: mysql:5.7
    networks:
      my_network: null
    restart: always
    volumes:
      - type: volume
        source: db_data
        target: /var/lib/mysql
        volume: {}
  wordpress2:
    depends_on:
      db2:
        condition: service_started
    environment:
      WORDPRESS_DB_HOST: db2:3306
      WORDPRESS_DB_NAME: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
      WORDPRESS_DB_USER: wordpress
    image: wordpress:latest
    networks:
      my_network: null
    ports:
      - mode: ingress
        target: 80
        published: "8082"
        protocol: tcp
    restart: always
networks:
  my_network:
    name: wordapp_my_network
volumes:
  db_data:
    name: wordapp_db_data
```

7.Pour consulter les logs des services en temps réel, exécute la commande **docker compose logs** :

Exemple 2 : Déploiement de Services avec Docker Compose : Personnalisation via Dockerfile

Déploiement de Services avec Docker Compose : Personnalisation via Dockerfile et Utilisation d'une Image Officielle

Objectif :Créer deux services avec Docker Compose :

- Un service utilisant l'image Docker officielle hello-world.
- Un service basé sur un Dockerfile qui affiche "Goodbye, Docker!".

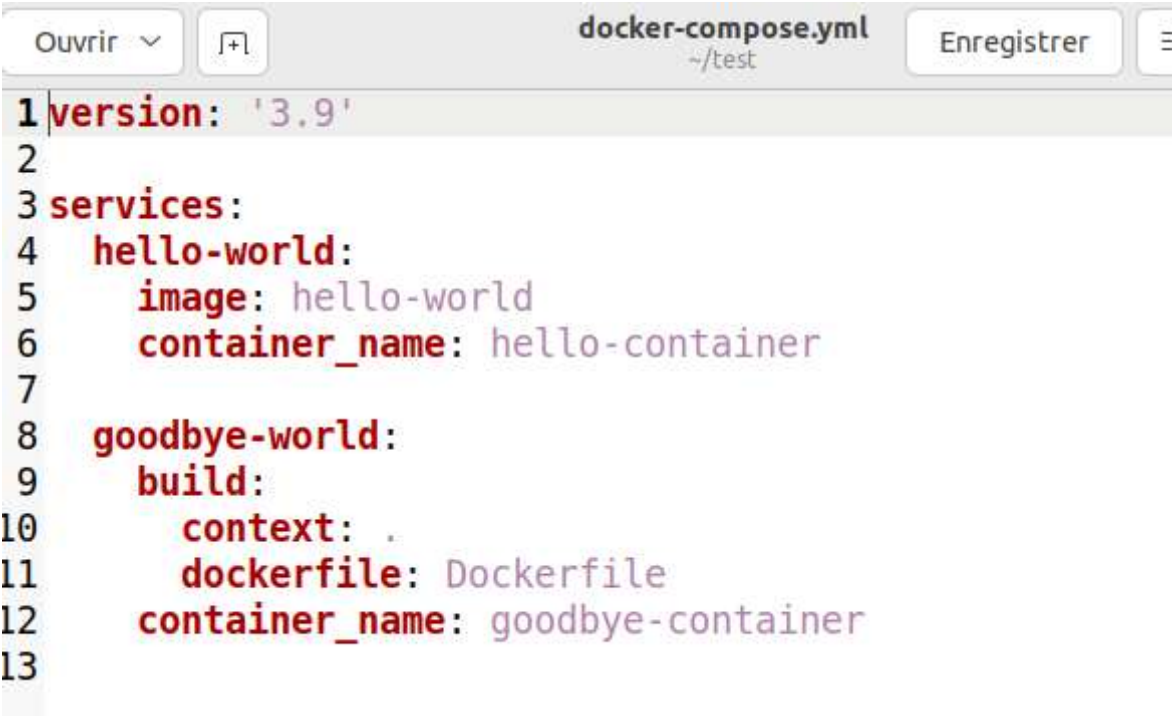
Instructions:

1.Créer un répertoire de travail nommé test:

```
esprit@esprit-virtual-machine:~$ mkdir test
esprit@esprit-virtual-machine:~$ cd test
```

2.Créer les deux fichier **Dockerfile** et **docker-compose.yml** sous le répertoire de travail:

Fichier docker-compose.yml:



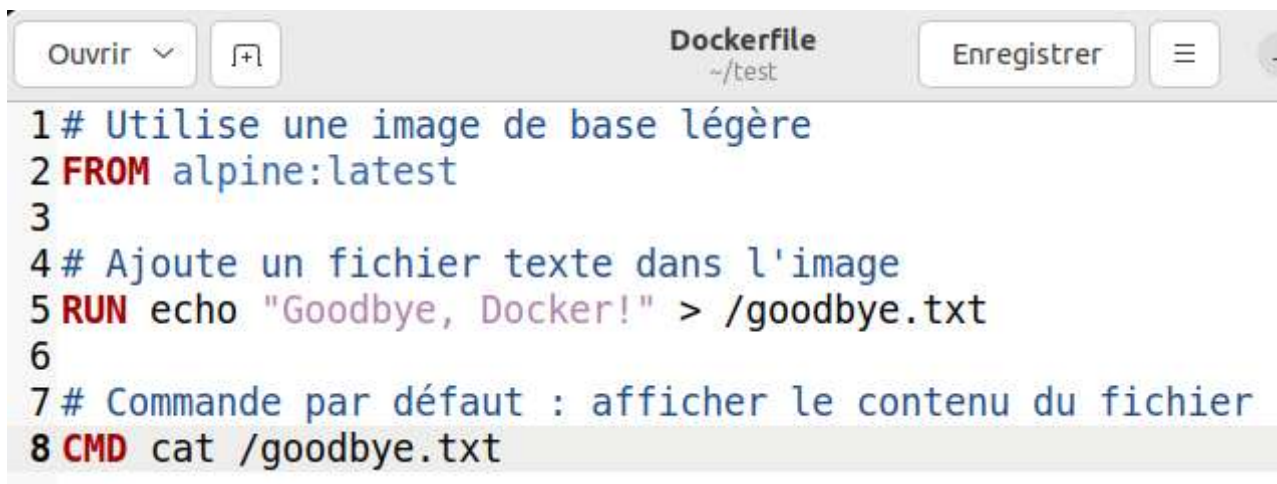
```
Ouvrir  [+]
```

docker-compose.yml
~/test

Enregistrer

```
1 version: '3.9'
2
3 services:
4   hello-world:
5     image: hello-world
6     container_name: hello-container
7
8   goodbye-world:
9     build:
10      context: .
11      dockerfile: Dockerfile
12     container_name: goodbye-container
13
```


Fichier Dockerfile pour le service goodbye-world



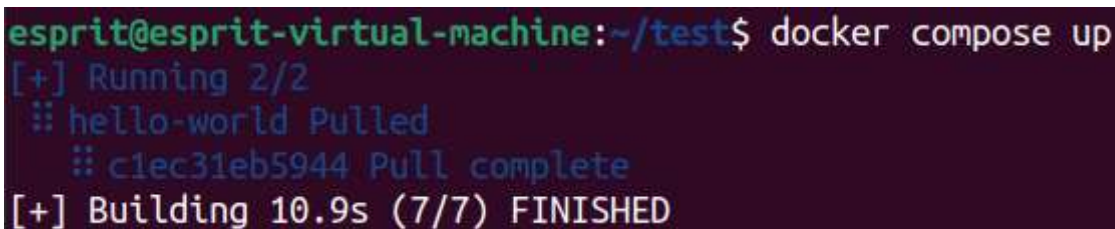
```
Ouvrir  Dockerfile  Enregistrer
~/test

1 # Utilise une image de base légère
2 FROM alpine:latest
3
4 # Ajoute un fichier texte dans l'image
5 RUN echo "Goodbye, Docker!" > /goodbye.txt
6
7 # Commande par défaut : afficher le contenu du fichier
8 CMD cat /goodbye.txt
```

Explications

- **Fichier docker-compose.yml :**
 - Définit deux services : goodbye-world et hello-world.
 - Le service goodbye-world utilise un Dockerfile situé dans le répertoire courant.
 - Le service hello-world utilise l'image Docker officielle hello-world, sans avoir besoin d'un fichier Dockerfile.
 - Chaque service a un nom de conteneur personnalisé : goodbye-container et hello-container.
- **Fichier Dockerfile pour goodbye-world :**
 - Utilise l'image de base Alpine.
 - Crée un fichier goodbye.txt contenant le texte "Goodbye, Docker!".
 - La commande par défaut affiche le contenu du fichier goodbye.txt.
- **Image Docker hello-world :**
 - Utiliser l'image officielle hello-world, qui affiche un message simple à propos de Docker lors du démarrage du conteneur.

3. Exécutez la commande **docker compose up** pour lancer les conteneurs :



```
esprit@esprit-virtual-machine:~/test$ docker compose up
[+] Running 2/2
  :: hello-world Pulled
  :: c1ec31eb5944 Pull complete
[+] Building 10.9s (7/7) FINISHED
```



```
## Container hello-container Created
## Container goodbye-container Created
Attaching to goodbye-container, hello-container
hello-container |
hello-container | Hello from Docker!
hello-container | This message shows that your installation appears to be working correctly.
hello-container |
hello-container | To generate this message, Docker took the following steps:
hello-container | 1. The Docker client contacted the Docker daemon.
hello-container | 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
hello-container |    (amd64)
hello-container | 3. The Docker daemon created a new container from that image which runs the
hello-container |    executable that produces the output you are currently reading.
hello-container | 4. The Docker daemon streamed that output to the Docker client, which sent it
hello-container |    to your terminal.
hello-container |
hello-container | To try something more ambitious, you can run an Ubuntu container with:
hello-container | $ docker run -it ubuntu bash
hello-container |
hello-container | Share images, automate workflows, and more with a free Docker ID:
hello-container | https://hub.docker.com/
hello-container |
hello-container | For more examples and ideas, visit:
hello-container | https://docs.docker.com/get-started/
hello-container |
goodbye-container | Goodbye, Docker!
hello-container exited with code 0
goodbye-container exited with code 0
```

Résultat attendu

- Le conteneur hello-container utilise l'image hello-world et affiche le message de bienvenue